Deployment & Monitoring Note

D4 - 420-977-VA AI Development Project - Manish Chauhan - Repo: AI-DS-Git/boreal-marche-customer-intelligence - AWS ca-central-1

1. What is deployed

The retention model (Head A), the basket recommender (Head B) and a tool-using assistant (Head C) with a rule provider that is live and OpenAI and Ollama providers that are implemented but not enabled on the deployed service are served by a FastAPI service and a bilingual (EN/FR) Streamlit UI. Both run as containers on AWS ECS Express Mode (Fargate behind a shared Application Load Balancer, HTTPS). Every push to *main* that passes CI is built, pushed to ECR with the commit SHA as the tag, deployed, and smoke-tested automatically. Live URLs are below; both were exercised from outside AWS (UI in a browser, API with the traffic simulator).

Item	Value
Live UI	https://bo-18e314a8dc3544148369c0305041cb8f.ecs.ca-central-1.on.aws
Live API (docs)	https://bo-ef5fe6766e224e0c8c335d5b98addb0c.ecs.ca-central-1.on.aws/docs - also /health, /ready, /version, /metrics
Deployed commit	772594e at time of writing; GET /version returns the same git_sha
Model	Extra Trees, isotonic-calibrated, 28 features, threshold 0.82; held-out PR-AUC 0.8753, ROC-AUC 0.7301
Repo tag	v0.9 (to be created on the final commit)

Rubric area	Where it is satisfied
Live system (20)	Section 1-2: public HTTPS URLs, health/readiness, version = commit SHA
API quality (15)	Section 3: validated schemas, uniform errors, request IDs, tests
Containers (15)	Section 4: multi-stage, non-root, healthcheck, compose
CI (15)	Section 5: lint, types, tests with coverage gate, Docker build, secret scan
CD (15)	Section 6: OIDC, ECR, ECS Express, post-deploy smoke test, human-gated retraining
Monitoring (20)	Section 7: drift, data quality, ops metrics, alerts, delayed ground truth

2. Architecture

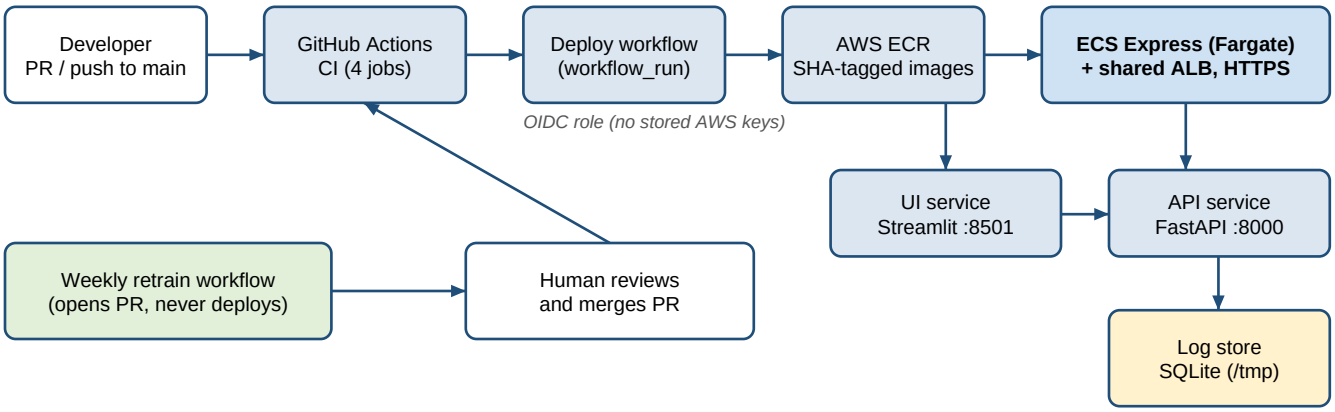


Figure 1. Delivery path and runtime topology. The yellow box marks the main limitation (Section 8): monitoring state lives in container-local SQLite.

The UI calls the API over HTTPS using the API_URL setting; the API loads one joblib artifact at start-up (preprocessing, estimator and calibrator are bundled so serving code cannot apply mismatched preprocessing). Predictions and requests are logged to a log store used by the monitoring endpoints. In local development the same images run with Docker Compose, where the log store is PostgreSQL.

3. API quality

Endpoint	Purpose
GET /health, /ready	Liveness vs readiness (ready only when the model and rules loaded)
GET /version	git_sha, model_version, decision threshold (0.82), rules count
POST /predict, /predict/batch	Retention probability, decision (target with offer if P(retain) < 0.82), explanation
POST /recommend	Association-rule basket recommendations (Head B)
POST /chat	Assistant answers grounded in the project rules and model facts
GET /monitoring/summary, POST /monitoring/outcomes, GET /metrics	Drift and ops summary; delayed-label ingestion; Prometheus-style metrics

- Pydantic v2 schemas with *extra=forbid*: unknown or malformed fields get a 422, never a silent default.
- Uniform error JSON and an *X-Request-ID* on every response, carried into the logs for traceability.
- Golden-prediction regression: customer 12347 must score 0.8095 (target with offer) - a test fails if the artifact or preprocessing drifts.
- 24 plain-Python tests plus FastAPI contract tests run in CI (coverage gate 70%).

Assistant (Head C). POST /chat and POST /chat/stream (server-sent events) serve three providers behind one interface: a deterministic rule provider (live), and OpenAI and Ollama providers with tool calling and token streaming, enabled only when a key or base URL is configured. Policy questions use embedding retrieval over the bilingual policies with an evidence floor; below it the assistant refuses and hands off to staff. Four typed tools are the only source of policy text, product codes and recommendations, privacy and out-of-scope requests are refused before any model sees them, and a verifier replaces any model answer containing a product code or percentage that no tool returned. A 40-question golden set (EN/FR) is scored in CI offline: tool choice 97.5%, refusals 97.5%, citations 95% (in-sample; the evidence floor was set on the same questions). The provider code is tested over real sockets against a local fake OpenAI/Ollama server. Figure 2 shows the live site answering a French question in French.

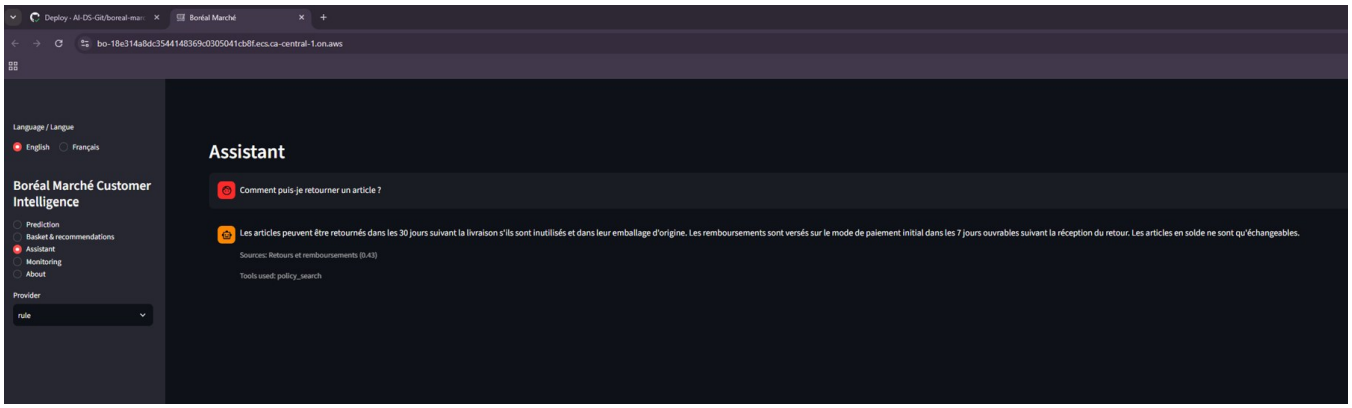


Figure 2. Live Assistant page: a French question gets a French answer with its source and tool.

4. Containers

- **Dockerfile (API)** and **Dockerfile.ui**: multi-stage, Python 3.11 slim, non-root user, HEALTHCHECK against /ready and /_stcore/health.
- scikit-learn 1.8.0, pandas 3.0.2, numpy 2.4.4 and joblib 1.5.3 are pinned exactly to the versions recorded in the model metadata (a mismatch can silently change predictions); web-stack packages use compatible ranges.
- **docker-compose.yml**: db (PostgreSQL), api, ui, plus an optional Ollama profile. *docker compose up --build* gives the full stack locally; this is the path a classmate uses for the cold-start test.
- Student's system Python is 3.9, below scikit-learn 1.8's minimum, so the container is also the supported way to run the API.

5. Continuous integration

Workflow *ci.yml* runs on every pull request and on push to main. Four jobs, each a separate status check:

Job	What it enforces
lint	ruff (E, F, W, B) and mypy on app/
test	pytest with 70% coverage gate: golden prediction, rules, assistant, monitoring maths, data-quality suite, API contract
docker	Builds both images; starts the API image and checks /ready and that /version equals the commit SHA
secret-scan	gitleaks over the full history

The pipeline went red-to-green during development on real findings (unused imports, duplicate set entries, six mypy typing errors), which is the intended behaviour of the gate. Branch protection on *main* requiring these four checks is configured as the last step before tagging v0.9 (status: <enabled / pending> at submission).

6. Continuous delivery

- **Trigger:** *deploy.yml* runs on *workflow_run* after CI succeeds on main (and manually via workflow_dispatch). Code that fails CI is never deployed.
- **Authentication:** GitHub OIDC to the IAM role *boreal-github-deploy*; no long-lived AWS keys exist anywhere. The trust policy pins the exact repository and the main branch.
- **Build:** images are tagged with the commit SHA and pushed to ECR; nothing deploys *latest*, so any release is reproducible and rollback is a re-run of a previous SHA.

- **Deploy:** official action *aws-actions/amazon-ecs-deploy-express-service* deploys the API (512 CPU / 1024 MB, health path /ready), the workflow waits until the live */version* reports the new SHA, then deploys the UI pointed at the API.
- **Verification:** a smoke test calls the live API and UI; the run summary prints both URLs. First successful run: Deploy #1, commit 772594e (about 11 minutes); Deploy #2, a manual run, took about 20.
- **Retraining (human-gated):** *retrain.yml* runs weekly, retrains with the frozen recipe and compares candidate and deployed model on the same held-out slice. A pull request is opened only if PR-AUC improves by at least 0.005 with no material loss in ROC-AUC (0.01) or Brier (0.005) and the feature schema and row count are intact. It never deploys; a human merges, and the merge triggers CI and then Deploy. The UCI dataset is static, so there is no real new signal: the first run in GitHub Actions scored the candidate at PR-AUC 0.8782 against 0.8764 for the deployed model, a gain of 0.0018, below the 0.005 gate, so it correctly kept the current model. That small gap is noise from tie-breaking at the split boundary (about 34 customers share the same last-purchase day at the 85% cut), which is why the gate demands a margin; the split is now deterministic (stable sort with customer ID tie-break). The job demonstrates the mechanism, not real improvement.

Deviations and lessons

- **App Runner replaced by ECS Express Mode.** The D2 plan named AWS App Runner. App Runner is closed to new customers, and Streamlit needs WebSockets, so the platform was changed to ECS Express Mode (Fargate + shared ALB), which supports both and still provides HTTPS and rolling deploys from one action.
- **OIDC subject format.** The first deploy failed at role assumption. GitHub's token subject now embeds owner and repository numeric IDs; CloudTrail showed the real subject, and the role trust policy was updated to the exact ID-based string. Loosening the policy with wildcards did not help and was reverted to the strict form.

7. Monitoring

Signal	Method	Thresholds
Input drift	PSI per feature against training reference (models/drift_reference.json) with sampling-noise-floor correction; KS statistic as a second view	amber 0.10, red 0.25
Prediction drift	PSI on the score distribution, reported separately from input drift	same bands
Data quality	Out-of-range share and null-rate delta per feature	traffic light
Operations	Request volume, p50/p95/p99 latency, error rate, business-rule fire rate	dashboard + /metrics
Outcomes	Delayed ground truth via POST /monitoring/outcomes; label delay is 90 days, so the first live accuracy signal is earliest prediction + 90 days	reported once labels arrive
Alerts	Red states raise a de-duplicated alert (cooldown) and call an optional webhook	configurable

Demonstration. scripts/simulate_traffic.py sends realistic customers from a 600-customer golden sample. Normal traffic shows green (max PSI about 0.08, p95 about 180 ms). With *--drift* (frequency x3-6, transaction_count x4, monetary_total x3-6) frequency_orders turns red (PSI 0.385) and monetary_total and transaction_count amber. The same demonstration was run against the live URL (Figure 3): drifted traffic gave red, and after a redeploy cleared the state, normal traffic gave green with 0.0% errors and p95 latency of 216 ms. Prediction drift moves very little under input shift, which is why the two are shown separately.

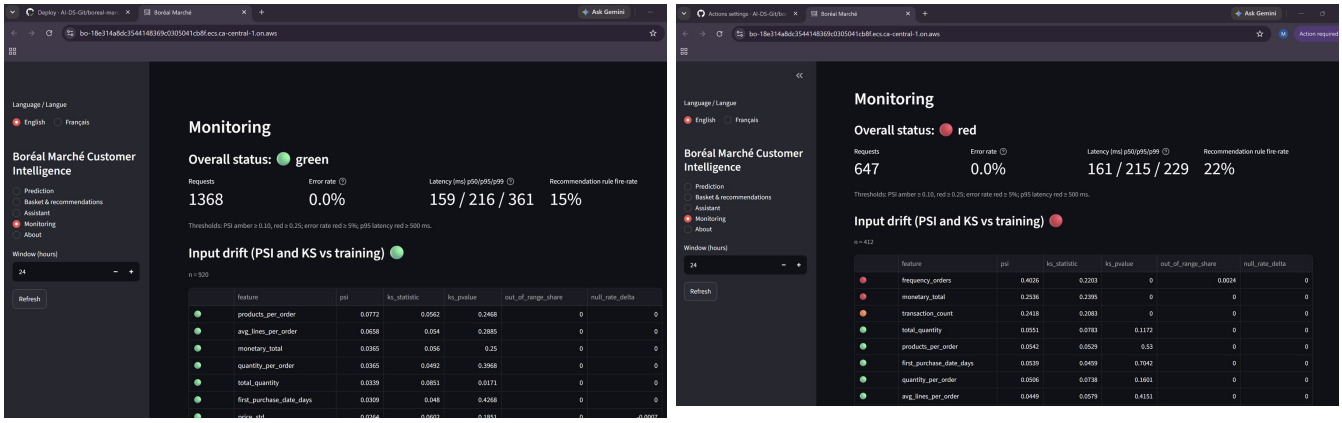


Figure 3. Live Monitoring page on the deployed UI. Left: after normal traffic (1,368 requests, n=920, all PSI under 0.10, green). Right: after drifted traffic (647 requests; frequency_orders and monetary_total red).

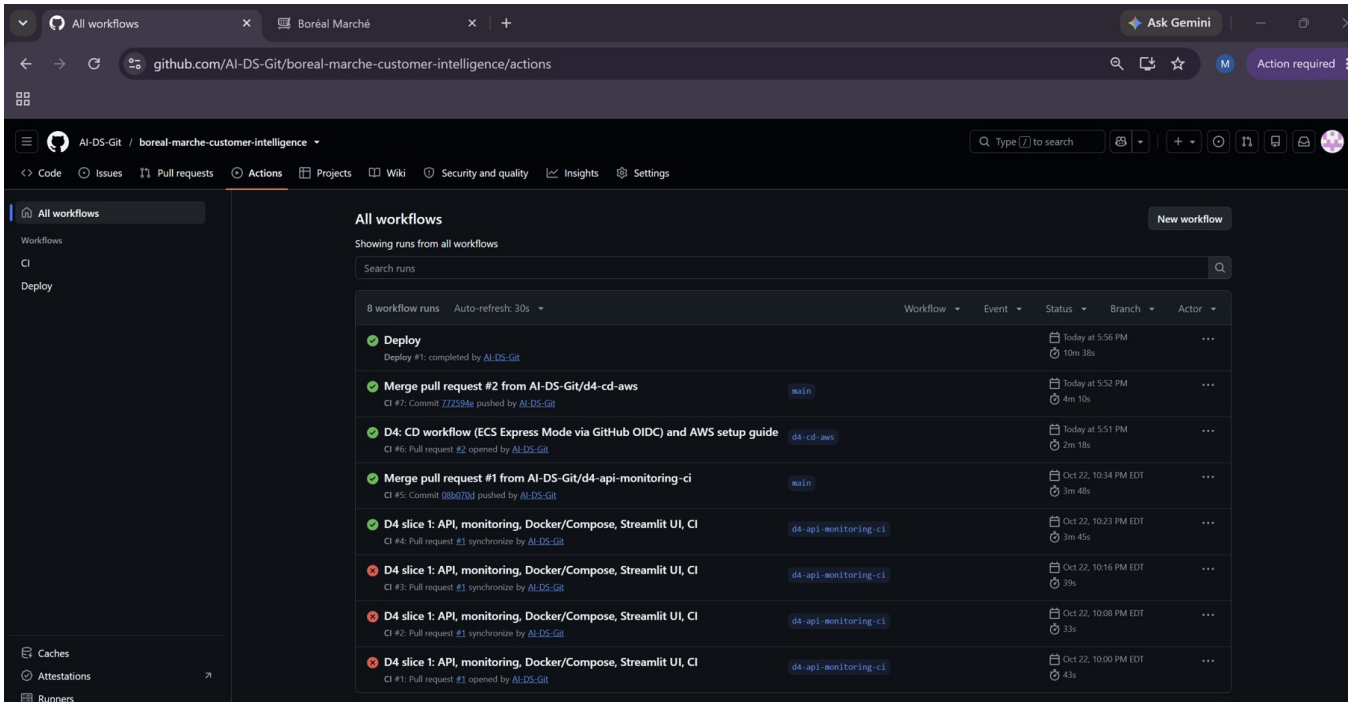


Figure 4. GitHub Actions history: CI red then green on PR #1, merge of PR #2 (CI #7) and Deploy #1 green; Deploy #2 (manual run) redeployed the same commit to reset state.

8. Known limitations and corrections

Item	Status and mitigation
Monitoring state is container-local	The deployed API logs to SQLite in /tmp, so history resets on each deploy or task restart and is not shared if the service scales out (min=max=1 for now). Mitigation: Postgres (proved in Compose) on RDS is the production path; not enabled to keep cost and complexity down.
Stale model in tag v0.3	While wiring the container I found the artifact committed at v0.3 was the earlier model (trained 2026-09-29, PR-AUC 0.8616, threshold 0.80), not the tuned model the D3 report describes (PR-AUC 0.8753, threshold 0.82). The correct artifact is on main from PR #1. The tag was left as submitted; D3 figures (PR-AUC 0.8753) are reproduced to within about 0.002 by the retraining script, the residual being the split tie-breaking noted in Section 6.

Item	Status and mitigation
Real LLMs not yet measured	The deployed service has no LLM key, so it serves the rule provider with offline hashing retrieval (lexical, not semantic). The OpenAI and Ollama providers, multilingual embedding retrieval and token streaming are implemented and tested only against a local fake server. Real-model tool choice and real-embedding quality are scored with scripts/eval_assistant.py and have not been run. Golden-set figures are in-sample.
Reproducibility	Base images are not yet digest-pinned and web-stack dependencies use ranges; a lock file is the next hardening step.
Policy content	Business rules and offer policies are prototype placeholders, as stated in D1-D3.

Cost and teardown. One Fargate task per service plus the shared ALB and ECR storage; an AWS budget with 80% and 100% alerts is active. After grading, delete the ECS Express services and the ECR repositories to stop billing.

Operating notes. Roll back by re-running Deploy on a previous commit SHA. Run the stack locally with *docker compose up --build*; run the drift demo with *python scripts/simulate_traffic.py --n 200 [--drift] [--api URL]*.